

Maismaasõidukite tehnoülevaatused Eestis (2023) - ETL Pipeline ja Visualiseerimine

Autorid: Mark-Kirill Gubal,

Aleksandr Markov,

Jaroslav Polovnikov,

Sergei Sizov

Antud projekti eesmärk oli luua modulaarne ja automatiseeritud ETL pipeline, mis võiks ise Ekstraktida andmed, Transformeerida need andmed OpenRefine'i sees, Laadida need puhastatud andmed üles SQLite andmebaasi ning visualiseerida puhastatud andmed RMarkdown formaadis.

Meie andmeallikaks osutus Andmete teabevärava andmed „Maismaasõidukite tehnoülevaatused Eestis“, täpsemalt selle andmed aastast 2023.

Maismaasõidukite tehnoülevaatused Eestis:

<https://andmed.eesti.ee/datasets/maismaasoidukite-tehnoulevaatused-eestis>)

Andmestiku API: <https://andmed.eesti.ee/api/datasets/ae47fec7-63d0-4b7a-969b-fbdfed21d52a/files/1943aed4-8e53-4e70-9946-7fc8ad1f7dfe/download-s3>

See andmestik hoiab andmed sõidukite kinnitatud tehnoülevaatuste tulemustest. Andmed on salvestatud CSV formaadis, kus iga rida on jaotatud järgmiselt:

Tabel 1. CSV faili tulbad

#	Päised	Andmetüübid	Tähendus
0	ID	puudub	
1	TEHNOYLEVAATUSPUNKT	str	Tehnoülevaatuspunkti nimi
2	PUNKTI_KOOD	str	Tehnoülevaatuspunkti kood
3	TOOTAJA	int	Töötaja number
4	YV_KUUPAEV	date	Ülevaatuse kuupäev (aasta ja kuu)
5	YLEVAATUSLIIK	str	Ülevaatuse liik
6	YLEVAATUSOTSUS	str	Ülevaatuse otsus
7	RIKKED	str	Rikked, mida tuvastati autol ülevaatuse
8	SOIDUK_ID	int	Sõiduki ID
9	ESMANE_REG_AASTA	int	Sõiduki esmane registreerimise aasta

10	MARK	str	Sõidukimark
11	MUDEL	str	Sõidukimudel
12	KATEGOORIA	str	Sõidukikategooria
13	KERETYYP	str	Sõiduki keretüüp

Kogu ETL protsess on rakendatud Dockeris ja seda saab kasutada kahel viisil:

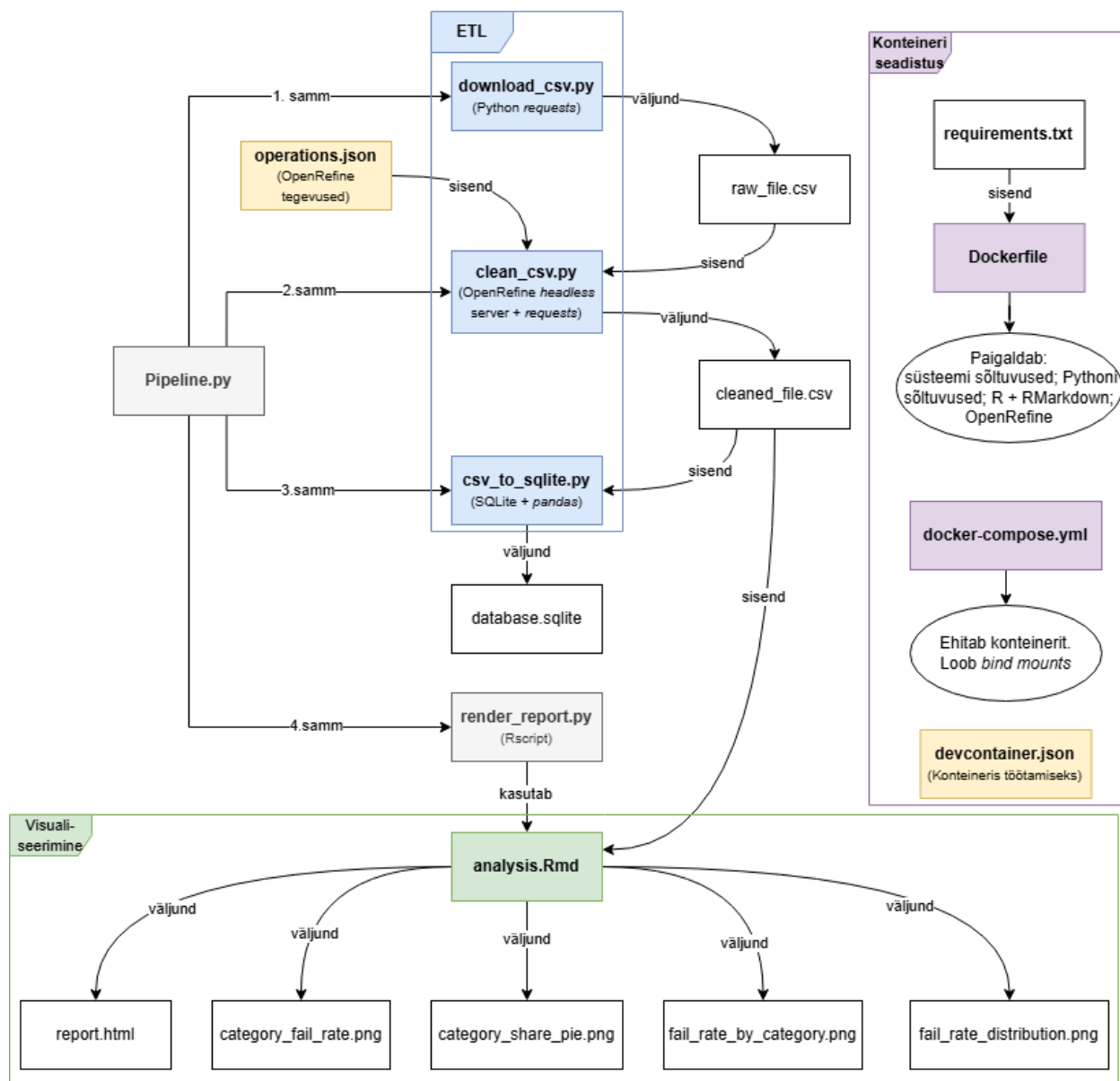
1. Kasutades Docker Compose, mis kohe pärast konteineri ehitamist, käivitab kogu pipeline'i ja visualiseerimist. Selle tulemusel salvestatakse kõik väljundid(CSV failid, SQLite andmebaas, visualiseerimise raport ja pildid) hostmasinas
2. Kasutades Dev Container, et töötada konteineri sees. See lähenemine võimaldab kasutajal ise valida, millised skriptid jooksutada ning väljundid ei salvesta automaatselt hostmasinas, vaid kasutaja ise peab valima ja salvestama seda, mida tahab.

Kuna Dockerfile paigaldab konteineris kõik vajalikud sõltuvused ja programmid ära, siis kasutajal on vaja paigaldada endale ainult Docker, Docker Compose ja Visual Studio Code koos Remonte – Containers laiendusega, kui too soovib kasutada Dev Container variandi.

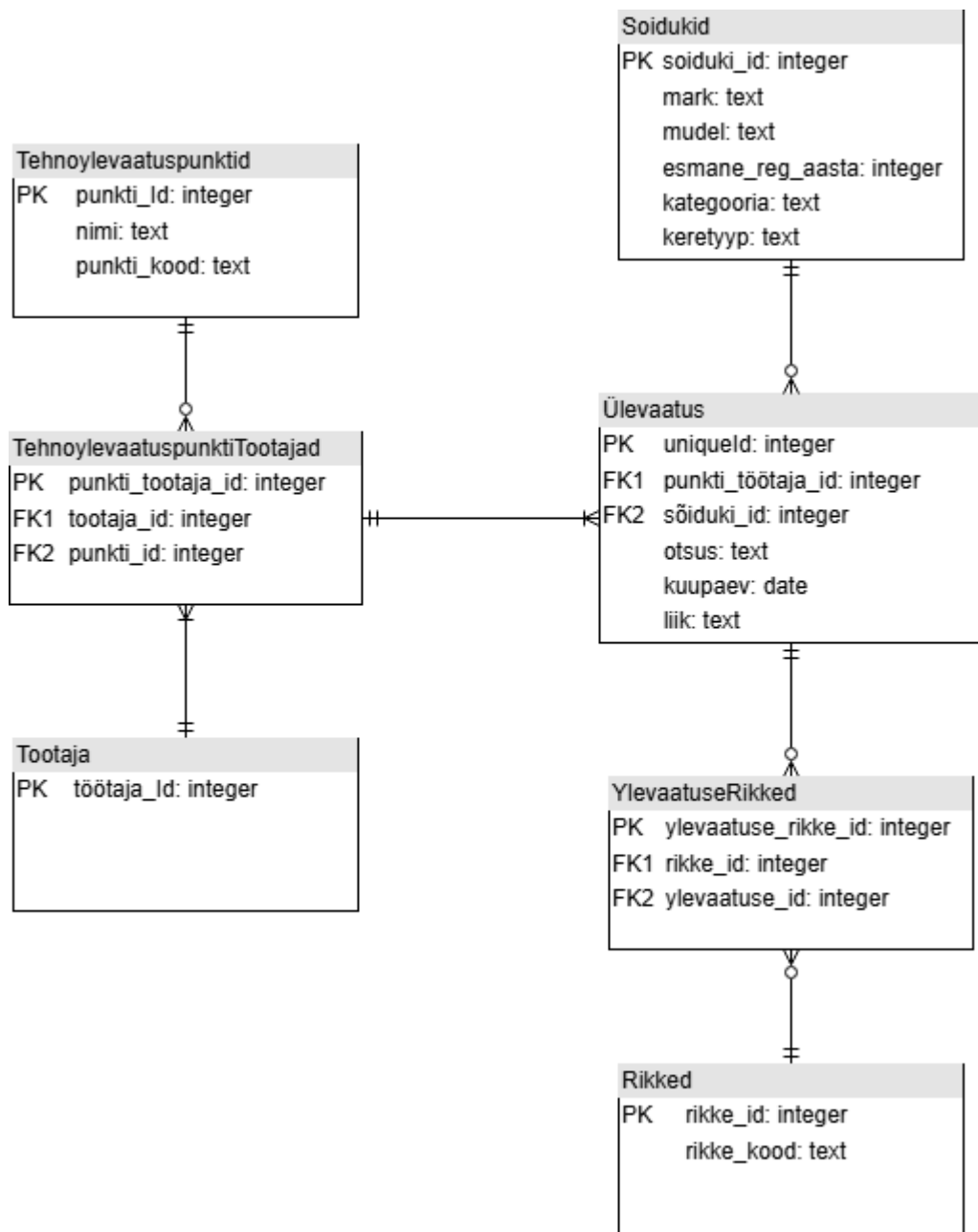
Projekti kogu protsessi ja ülesehitus võib jaotada järgmiselt:

1. Docker Compose või Dev Container ehitab konteinerit
2. Konteineris paigaldatakse kõik ära
3. **Orkestreerija**(pipeline.py) jooksub:
 - 3.1. **Extract** (download_csv.py) skripti, mis laeb alla andmed API-st kasutades Python *requests*
 - 3.2. **Transform** (clean_csv.py) skripti, mis kasutab OpenRefine'i, et puhastada andmed
 - 3.3. **Load and Transform** (csv_to_sqlite.py) skripti, mis ehitab üles SQLite andmebaasi ja laeb selle sisse puhastatud andmed. See skript teeb veidi ELT-i juba, kuna ühes tabelis (YlevaatuseRikked) umbes 0,025% ridades on NULL väärtused.
4. **Visualiseerimise** (render_report.py) skripti, mis kasutab analysis.Rmd raporti ja piltide genereerimiseks. Analysis.Rmd kasutab R-i ja ggplot2-d sõidukite tehnoülevaatuse andmete visualiseerimiseks, kuvades kategooriate jaotusi, ülevaatuspunktide koormust, ebaõnnestumismäära ja erandid.

Projekti ülesehitus: ETL Pipeline, Visualiseerimine, Docker



Joonis 1. Projekti ülesehitus



Joonis 2. SQLite andmebaasi ERD

Projekti juures kõige raskemaks osutus OpenRefine automatiseerimine. Esiteks, me pidime puhastama käsitsi toores CSV faili, et salvestada tuvastada ja salvestada kõik vajalikud tegevused (Dokumendi täpsematest tegudest saab leida projekti GitHubis). Teiseks, me pole varem tegelenud OpenRefine automatiseerimisega, seega see: kuidas selle paigaldada konteineris; kuidas seda käivitada *headless* režiimis; mis on selle CSRF token jne oli meie jaoks uus. Kolmandaks, me ei suutnud kaua aega lahendada viga „Missing or invalid CSRF token“, kuid meil pidi kõik korras olema. Tuli lõpuks välja, et tegu oli mitte CSRF otseselt, vaid meie andmete fail oli liiga suur, seega oli vaja anda OpenRefine’ile rohkem mälu.

Veel ebamugavaks osutus SQLite andmebaasis tabelid Rikked ja Ylevaatuserikked. CSV failis olid ühe ülevaatuserikke salvestatud koos ühes atribuudis, seega neid pidi eraldama teineteist ning leida võimalust, kuidas seostada neid vastavate ülevaatusetega.

See seostus ei ole lihtne, kuna teoorias sama ülevaatuspunkti töötaja saab samal kuupäeval(aasta+kuu) kontrollida sama autot samal ülevaatusel liigil ja anda sama otsust. Seda sai lahendatud nii, et meie kasutasime CSV rea numbrit, et seostada ülevaatus ja selle rikkeid.

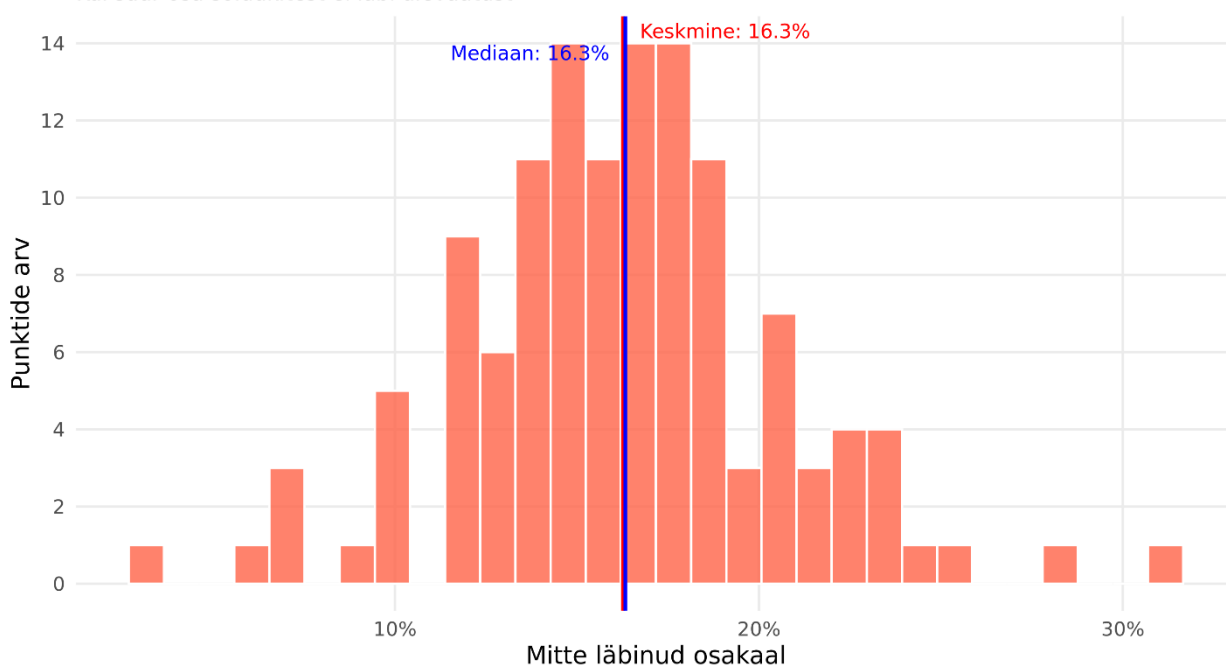
Kokkuvõtteks sai meil valmis:

- Dockeris rakendatud automatiseeritud pipeline ja visualiseerimine
- Visualiseerimise raport koos kujutistega
- Modulaarne protsess: projekt on jaotatud skriptideks, mida saab muuta või vahetada välja, peamiselt et skriptidel olid sarnased väljundid

Projekt on praegu mõeldud just meie poolt valitud CSV failile, ent kui vahetada API teiseks aastaks ja muuta operations.json(kas üldistada seda või spetsialiseerida teisele aastale), siis antud projekt saaks hakkame iga „Maismaasõidukite tehnoulevaatused Eestis“ andmestiku aastaga.

Ülevaatuspunktide rangeuse jaotus

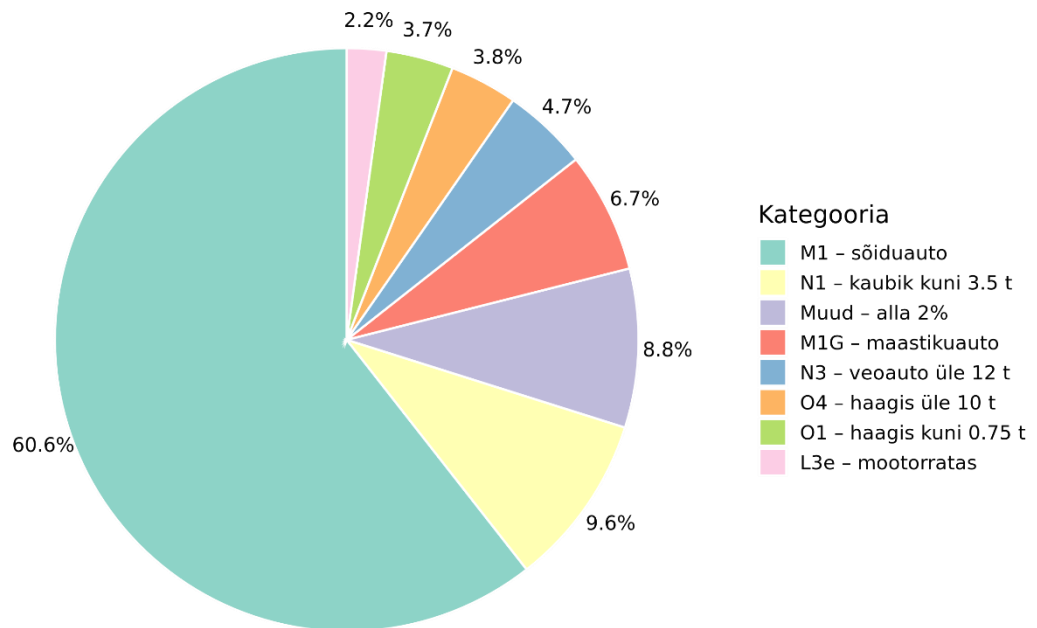
Kui suur osa sõidukitest ei läbi ülevaatus



Joonis 3. Näide visualiseerimisest: Ülevaatuspunktid rangeuse jaotus

Sõidukikategooriate osakaal

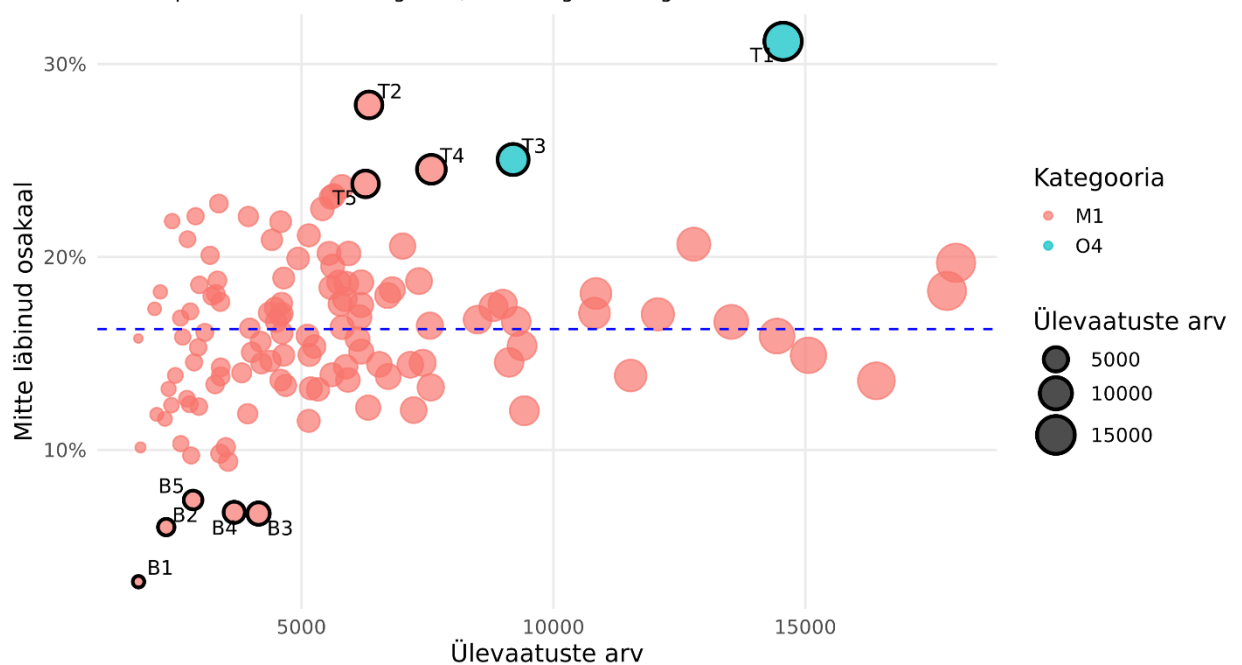
Alla 2% kategooriad on ühendatud gruppi 'Muud'



Joonis 4. Näide visualiseerimisest: Sõidukikategooriate osakaal

Ülevaatuspunktide rangeus vs koormus

Värv = peamine sõidukikategooria; must ring = 5 rangeimat / 5 leebeimat



Joonis 5. Näide visualiseerimisest: Ülevaatuspunktide rangeus vs koormus